1. INTRODUCTION

Security and surveillance systems play an important role in protecting restricted and sensitive areas such as industrial facilities, warehouses, campuses, laboratories, and other controlled environments. Conventional surveillance systems mainly rely on Closed-Circuit Television (CCTV) cameras that continuously record video footage. Although these systems provide visual monitoring, they require constant human supervision to detect suspicious activities or unauthorized access. Continuous monitoring by security personnel can often lead to fatigue, distraction, and delayed responses, which increases the chances of missing critical intrusion events. With the rapid advancement of Artificial Intelligence (AI) and computer vision technologies, modern surveillance systems are becoming more intelligent and automated. AI-based surveillance systems can analyze video frames, detect objects, and interpret human behavior without the need for continuous human observation. This project presents an AI-Based Intrusion Prediction System designed to enhance surveillance efficiency by automatically detecting and predicting human intrusion events. The system uses the YOLOv8 deep learning object detection model to identify and track individuals in video footage. A restricted area is defined using a polygon-based boundary, and the system continuously analyzes the movement of detected individuals relative to this boundary. By evaluating factors such as distance from the restricted area, movement direction, and approach consistency over time, the system calculates a risk level that predicts the likelihood of intrusion before the boundary is crossed. When an intrusion occurs, the system captures a screenshot and sends an automated email alert to the authorized user. This intelligent approach improves surveillance accuracy, reduces dependency on manual monitoring, and enables faster response to potential security threats.

1.1 OVERVIEW OF THE PROJECT

The AI-Based Intrusion Prediction System is designed to automatically monitor restricted areas and detect suspicious movement patterns using computer vision and deep learning techniques. The primary goal of the system is to predict possible intrusion attempts before they actually occur, allowing security personnel to respond more quickly and effectively.

The system operates using video input obtained from a surveillance camera or recorded footage. Each video frame is processed sequentially to detect human presence and analyze movement behavior. The detection process is performed using a deep learning object detection model, specifically the YOLOv8 algorithm, which is capable of identifying people in real time with high accuracy.

Once individuals are detected within the video frame, the system assigns unique identification numbers to track them across multiple frames. This tracking mechanism enables the system to analyze the movement trajectory of each person and determine whether they are approaching the restricted area.

A polygon-based boundary is used to define the restricted zone within the camera view. This boundary can be customized depending on the environment being monitored. The system continuously calculates the distance between detected individuals and the restricted boundary.

If a person moves toward the restricted area consistently over several frames, the system increases the predicted risk level. This risk value indicates the likelihood that the individual may attempt to enter the restricted zone. The risk percentage is displayed in real time on the monitoring screen.

When the individual crosses the restricted boundary, the system confirms the intrusion event. At this stage, the system captures an image of the frame and automatically sends an email notification to the authorized user. This notification includes the captured image as evidence of the intrusion event.

The system also maintains an intrusion counter that records the total number of detected intrusion events during system operation. This information can be used for monitoring and analysis purposes.

By combining object detection, tracking, behavioral analysis, and automated alerts, the system provides an intelligent surveillance solution that improves security monitoring while reducing the need for manual observation.

1.2 MODULE DESCRIPTION

The AI-Based Intrusion Prediction System is divided into several modules that work together to detect, analyze, and alert intrusion events in restricted areas.

1. Video Input Module

This module captures video frames from the input source such as a surveillance camera or recorded video. These frames are continuously processed by the system for human detection and monitoring.

2. Preprocessing Module

The preprocessing module prepares the input frames for analysis by resizing and normalizing them. This improves detection efficiency and ensures smooth processing of video frames.

3. Human Detection Module

This module uses the YOLOv8 deep learning model to detect human presence in each video frame. Detected persons are highlighted using bounding boxes for easy identification.

4. Object Tracking Module

The tracking module assigns unique IDs to detected individuals and tracks their movement across frames. This helps analyze behavior and movement patterns over time.

5. Intrusion Prediction Module

This module analyzes movement patterns such as distance from the boundary and direction of movement. Based on this analysis, the system predicts the probability of intrusion.

6. Risk Analysis Module

The system calculates a risk level for each detected person using movement behavior and proximity to the restricted area. This risk score indicates the likelihood of intrusion.

7. Alert Generation Module

When an intrusion is detected, this module captures a screenshot and sends an automated email notification to the authorized user.

8. Output Display Module

This module displays real-time results including bounding boxes, risk percentage, intrusion status, and total intrusion count on the monitoring screen.

2. SYSTEM ANALYSIS

System analysis is an important stage in system development where the current system is studied and evaluated in order to understand its limitations and identify opportunities for improvement. It involves examining existing surveillance solutions, identifying their weaknesses, and designing a better system to overcome those limitations.

In the context of surveillance and security monitoring, traditional systems primarily rely on CCTV cameras that continuously record video footage. These systems are widely used in industries, educational institutions, transportation facilities, and other public areas to monitor activities and prevent unauthorized access.

Although these surveillance systems provide visual monitoring, they largely depend on human operators to observe video feeds and identify suspicious behavior. This reliance on manual observation introduces several challenges such as delayed response, human error, and inefficient monitoring of multiple cameras.

With the rapid advancement of Artificial Intelligence and Computer Vision technologies, modern surveillance systems are shifting toward intelligent automation. AI-based systems can automatically analyze video frames, detect objects, and track human movement without constant human supervision.

System analysis for this project focuses on understanding the limitations of traditional surveillance systems and proposing an improved system that integrates deep learning techniques to detect and predict intrusion events automatically.

The proposed system introduces an AI-driven approach where human movement patterns are analyzed relative to a restricted boundary. Instead of detecting intrusion only after it occurs, the system predicts the likelihood of intrusion in advance, allowing security personnel to take preventive action.

2.1 EXISTING SYSTEM

Existing surveillance systems mainly rely on CCTV cameras installed at strategic locations to monitor restricted areas. These cameras continuously capture video footage, which is either stored for later review or observed in real time by security personnel.

In most environments, security staff are responsible for watching multiple video streams displayed on monitoring screens. Their task is to identify suspicious activities or unauthorized entry into restricted areas. If an intrusion occurs, the security personnel must manually respond by notifying authorities or taking necessary action.

Although CCTV systems provide visual evidence of events, they lack the ability to automatically interpret the activities occurring within the monitored environment. As a result, these systems are limited to passive monitoring and cannot actively analyze behavior patterns or predict potential threats.

In large surveillance environments where many cameras are installed, monitoring becomes even more difficult. Security personnel may miss important events due to fatigue or distraction, especially when monitoring screens for long durations.

Another limitation of traditional surveillance systems is that they only provide information **after an event has already occurred**. For example, intrusion alerts may only be triggered when a restricted boundary has been crossed. This reduces the available response time and increases the risk of damage or security breaches.

Furthermore, manual monitoring requires significant human resources, increasing operational costs for organizations that must employ multiple security personnel to monitor surveillance systems continuously.

Due to these limitations, there is a growing need for intelligent surveillance systems that can automatically detect suspicious behavior and generate alerts without relying entirely on human observation.

2.1.1 DISADVANTAGES OF EXISTING SYSTEM

The existing surveillance systems suffer from several disadvantages that limit their effectiveness in modern security environments.

1. Dependence on Human Monitoring

Traditional CCTV systems require continuous monitoring by human operators. Security personnel must constantly observe video feeds to detect suspicious activities. This process is time-consuming and highly dependent on human attention.

2. Human Error and Fatigue

Monitoring multiple screens for extended periods can cause fatigue and reduced concentration. As a result, operators may overlook important events or respond too late to intrusion attempts.

3. Lack of Predictive Capability

Existing systems generally detect intrusions only after a person has already entered a restricted area. These systems do not analyze movement patterns to predict potential intrusion in advance.

4. Delayed Response to Security Threats

Since intrusion detection relies on manual observation, the response time may be slow. Delayed detection can allow unauthorized individuals to access sensitive areas before action is taken.

5. Difficulty in Monitoring Multiple Cameras

Large surveillance setups often involve many cameras. Monitoring all video feeds simultaneously can be overwhelming for security personnel, increasing the chances of missing critical events.

6. High Operational Cost

Continuous monitoring requires multiple security staff working in shifts, which increases operational costs for organizations.

7. Limited Intelligence

Traditional systems mainly record video footage but lack intelligent analysis capabilities. They cannot automatically detect suspicious behavior or generate predictive alerts.

2.2 PROPOSED SYSTEM

To overcome the limitations of traditional surveillance systems, this project proposes an **AI-Based Intrusion Prediction System** that uses computer vision and deep learning techniques to automatically monitor restricted areas and predict potential intrusion events.

The proposed system integrates an object detection model capable of identifying human presence in video frames. The system uses the **YOLOv8 deep learning algorithm**, which is known for its high detection accuracy and real-time performance.

In this system, a restricted zone is defined within the surveillance frame using a polygon-based boundary. The system continuously analyzes the movement of detected individuals relative to this boundary.

Each detected person is assigned a unique ID and tracked across multiple frames. This tracking mechanism allows the system to analyze movement trajectories and determine whether an individual is approaching the restricted area.

Instead of detecting intrusion only after the boundary is crossed, the proposed system evaluates several behavioral parameters such as:

- Distance from the restricted boundary
- Direction of movement
- Consistency of approach across frames

Based on these parameters, the system calculates a **risk level representing the probability of intrusion**. When a person moves closer to the restricted zone over time, the predicted risk value increases.

If the person eventually enters the restricted area, the system confirms the intrusion event and immediately generates an alert.

The alert system automatically captures a screenshot of the frame and sends an **email notification to the authorized user** using the SMTP protocol. This ensures that security personnel are informed about the intrusion event in real time.

The system also provides visual feedback on the monitoring screen, including bounding boxes around detected individuals, predicted risk percentage, intrusion status, and total intrusion count.

By automating the detection and prediction process, the proposed system significantly reduces the need for continuous human supervision while improving the speed and accuracy of surveillance monitoring.

2.2.1 ADVANTAGES OF PROPOSED SYSTEM

The proposed AI-Based Intrusion Prediction System offers several advantages compared to traditional surveillance systems.

1. Automated Intrusion Detection

The system automatically detects human presence using deep learning algorithms, reducing the need for manual monitoring.

2. Intrusion Prediction Capability

Unlike traditional systems, the proposed system can predict potential intrusion based on movement behavior before the restricted boundary is crossed.

3. Improved Security Response Time

Early risk prediction allows security personnel to take preventive action before an actual intrusion occurs.

4. Reduced Human Intervention

The system minimizes dependence on human operators by automating detection and alert generation.

5. Real-Time Monitoring

The system provides real-time visual feedback including bounding boxes, risk levels, and intrusion alerts.

6. Automated Alert Notifications

Email alerts with captured screenshots ensure that authorized users are immediately informed about intrusion events.

7. Cost Efficiency

By reducing the need for continuous human monitoring, the system lowers operational costs for organizations.

8. Scalability

The system can be extended in the future to support multiple cameras, cloud monitoring, and additional security features.

3. SYSTEM SPECIFICATION

System specification describes the hardware and software requirements needed to develop and run the AI-Based Intrusion Prediction System. It provides details about the development environment, programming language, libraries, and system configuration required for successful implementation.

The proposed system is implemented using Python programming language along with computer vision and deep learning libraries. These technologies enable efficient video processing, human detection, movement tracking, and automated alert generation.

The system is designed to operate on standard personal computers with moderate hardware requirements. It can run efficiently without the need for specialized hardware, although systems equipped with graphical processing units (GPUs) can achieve faster processing performance.

The software architecture integrates multiple modules including video processing, object detection, behavior analysis, and alert notification. These modules work together to provide an automated surveillance solution capable of detecting and predicting intrusion events in real time.

3.1 SOFTWARE SPECIFICATION

The software components used in the development of the AI-Based Intrusion Prediction System are listed below.

Operating System

Windows 10 / Windows 11 or Linux

Programming Language

Python

Development Environment

Visual Studio Code / PyCharm / Jupyter Notebook

Libraries and Frameworks Used

1. **OpenCV**

   OpenCV (Open Source Computer Vision Library) is used for video processing and image analysis. It provides functions for capturing video frames, resizing images, drawing bounding boxes, and displaying visual outputs.

2. **Ultralytics YOLOv8**
   YOLOv8 is a deep learning object detection model used to detect human presence in video frames. It provides high accuracy and real-time performance, making it suitable for surveillance applications.

3. **NumPy**
   NumPy is used for numerical operations and array processing. It helps in handling image data and performing calculations related to movement analysis.

4. **PyTorch**
   PyTorch is the deep learning framework used to run the YOLOv8 model. It enables efficient model execution and GPU acceleration when available.

5. **SMTP Library**
   The Simple Mail Transfer Protocol (SMTP) library is used to send automated email notifications when an intrusion is detected.

6. **Matplotlib (Optional)**
   Matplotlib may be used for visualization or debugging during system development.

These software tools provide the necessary functionalities for implementing object detection, tracking, and alert mechanisms within the system.

3.2 HARDWARE SPECIFICATION

The hardware requirements for running the AI-Based Intrusion Prediction System are relatively moderate. The system can run on a standard personal computer or laptop with the following configuration.

1. Computer / Laptop

A computer or laptop is required to develop and run the AI-Based Intrusion Prediction System. It is used to execute the Python program, process video frames, and display detection results. The system should have sufficient processing power and memory to handle computer vision tasks efficiently. A laptop with a webcam or a desktop computer connected to a camera can be used for monitoring and testing.

2. Processor : Intel Core i5 (or higher)

The processor is responsible for executing program instructions and performing computational tasks involved in video processing and object detection. A processor such as Intel Core i5, Intel Core i7, or equivalent is recommended to ensure smooth performance. Faster processors improve the system's ability to process video frames quickly and support real-time detection.

3. RAM : 8 GB

Random Access Memory (RAM) is required to store and process temporary data while the program is running. For this project, a minimum of 8 GB RAM is recommended to ensure smooth execution of the Python program and computer vision libraries. Adequate RAM allows the system to process video frames without delays and improves overall system performance.

4. Input Device : Webcam / Camera

Input devices provide video data to the system for processing. In this project, a webcam or surveillance camera is used to capture video frames of the monitored area. These frames are analyzed by the system to detect human presence and identify potential intrusion events. Recorded video files can also be used as input during testing.

5. Storage

Storage is required to install the operating system, development tools, and project files. It is also used to store recorded videos, captured screenshots of intrusion events, and system logs. A minimum of 256 GB storage is recommended to ensure sufficient space for the application and related data.

3.3 ABOUT THE SOFTWARE

This section provides a brief overview of the main software technologies used in the development of the system.

1. **Python**

   Python is a high-level programming language widely used in artificial intelligence and machine learning applications. It is known for its simplicity, readability, and extensive library support. Python allows developers to quickly implement complex algorithms and integrate various modules within a single application.

2. **OpenCV**

   OpenCV is an open-source computer vision library used for image and video processing. It provides powerful tools for capturing video streams, processing frames, detecting objects, and displaying graphical outputs. In this project, OpenCV is used for handling video input, drawing bounding boxes, and displaying real-time system output.

3. **YOLOv8**

   YOLOv8 (You Only Look Once version 8) is a state-of-the-art object detection algorithm developed by Ultralytics. It is designed for real-time object detection tasks and is capable of identifying multiple objects within a single frame. In this project, YOLOv8 is used to detect human presence in surveillance footage.

4. **PyTorch**

   PyTorch is a deep learning framework used to build and run machine learning models. It provides efficient tensor operations and supports GPU acceleration, making it suitable for training and deploying deep learning models such as YOLOv8.

5. **SMTP Protocol**

   The Simple Mail Transfer Protocol (SMTP) is used for sending automated email notifications. When the system detects an intrusion event, it captures a screenshot and sends an email alert to the authorized user. This feature ensures that security personnel receive immediate notification of intrusion events.

4. SYSTEM DESIGN

System design is an important stage in software development where the architecture and operational structure of the system are defined. It describes how different components of the system interact with each other to perform the required functions. The design phase converts the requirements identified during system analysis into a detailed plan for implementation.

The AI-Based Intrusion Prediction System is designed using a modular architecture that integrates several components including video processing, human detection, object tracking, behavior analysis, and alert generation. Each module performs a specific task and communicates with other modules to ensure smooth system operation.

The system begins by receiving video input from a surveillance source. Each frame from the video is processed using computer vision techniques to detect human presence. Detected individuals are tracked across frames, and their movement patterns are analyzed relative to a restricted area defined by the user.

If a person approaches the restricted boundary consistently, the system increases the predicted risk level. When the boundary is crossed, the system confirms an intrusion event and triggers an alert mechanism that captures an image and sends an email notification to the authorized user.

The system design ensures efficient processing of video frames while maintaining real-time monitoring capability. The following subsections describe the design of different system components.

4.1 INPUT DESIGN

Input design focuses on how the system receives and processes data required for its operation. In the AI-Based Intrusion Prediction System, the primary input is a video stream that contains visual information about the monitored environment.

The system processes video input frame by frame. Each frame is extracted from the video source and passed to the preprocessing stage. During preprocessing, frames are resized and normalized to ensure consistent resolution and improve detection efficiency.

After preprocessing, the frames are forwarded to the human detection module where the deep learning model identifies individuals present in the scene. The coordinates of detected persons are used to track their movement and analyze their proximity to the restricted area.

The input design ensures that the system can efficiently process continuous video frames without significant delay. This enables the system to monitor the environment in near real-time and detect suspicious movement patterns quickly.

Input design also allows flexibility in selecting the video source. The system can operate using recorded video footage for testing purposes and can be extended in the future to support live camera feeds.

4.2 DATABASE DESIGN

The AI-Based Intrusion Prediction System does not require a traditional database management system because the system processes video data in real time. Instead of storing large amounts of data permanently, the system uses temporary memory structures to maintain necessary information during execution.

The following types of data are maintained temporarily in the system:

- Person identification numbers assigned to detected individuals
- Coordinates of detected objects within each frame
- Movement history of tracked individuals
- Distance of each person from the restricted boundary
- Calculated risk values for intrusion prediction
- Intrusion count indicating the number of detected intrusion events

These data elements are stored in memory structures such as arrays or dictionaries while the system is running. This design reduces the need for large storage capacity and allows faster processing.

Captured images of intrusion events may be stored locally for record and analysis purposes. These images provide visual evidence that can be reviewed later if required.

The database design approach used in this system focuses on efficient real-time processing rather than long-term data storage.

***Table 1**: Person Detection Table*

Field Name	Data Type	Description
Person_ID	**Integer**	**Unique identifier assigned to each detected person**
Frame_Number	**Integer**	**Frame number where the person was detected**
Bounding_Box	**Coordinates**	**Position of detected person in the frame**
Detection_Time	**Timestamp**	**Time when the person was detected**

Table 2 : *Movement Tracking Table*

Field Name	Data Type	Description
Person_ID	**Integer**	**Unique ID of detected person**
X_Position	**Float**	**X coordinate of the detected person**
Y_Position	**Float**	**Y coordinate of the detected person**
Distance_From_Boundary	**Float**	**Distance between the person and restricted boundary**
Movement_Direction	**String**	**Direction of movement relative to boundary**

Table 3 : *Intrusion Event Table*

Field Name	Data Type	Description
Event_ID	**Integer**	**Unique ID for each intrusion event**
Person_ID	**Integer**	**ID of the detected person**
Risk_Level	**Float**	**Predicted risk percentage**
Intrusion_Status	**Boolean**	**Indicates whether intrusion occurred**
Screenshot_Path	**String**	**Path of captured image**
Alert_Status	**Boolean**	**Indicates whether alert email was sent**

4.3 SYSTEM FLOW DIAGRAM

The system flow diagram illustrates the sequence of operations performed by the AI-Based Intrusion Prediction System during execution. It shows how data moves through different processing stages from video input to alert generation.

The process begins when the system loads the YOLOv8 object detection model and initializes the required libraries. The system then reads frames from the video input source.

Each frame undergoes preprocessing before being passed to the human detection module. The detection module identifies individuals present in the frame and generates bounding boxes around detected persons.

After detection, the tracking module assigns unique identification numbers to individuals and monitors their movement across frames. The system then calculates the distance between each detected person and the restricted boundary.

Using this information, the system evaluates the movement direction and determines whether the person is approaching the restricted area. Based on this analysis, the system calculates a risk value representing the probability of intrusion.

If the risk value exceeds a certain threshold, the system indicates a high likelihood of intrusion. When a person enters the restricted boundary, the system confirms the intrusion event.

At this stage, the alert module captures a screenshot of the frame and sends an automated email notification to the authorized user. The system also updates the intrusion count and continues monitoring subsequent frames.

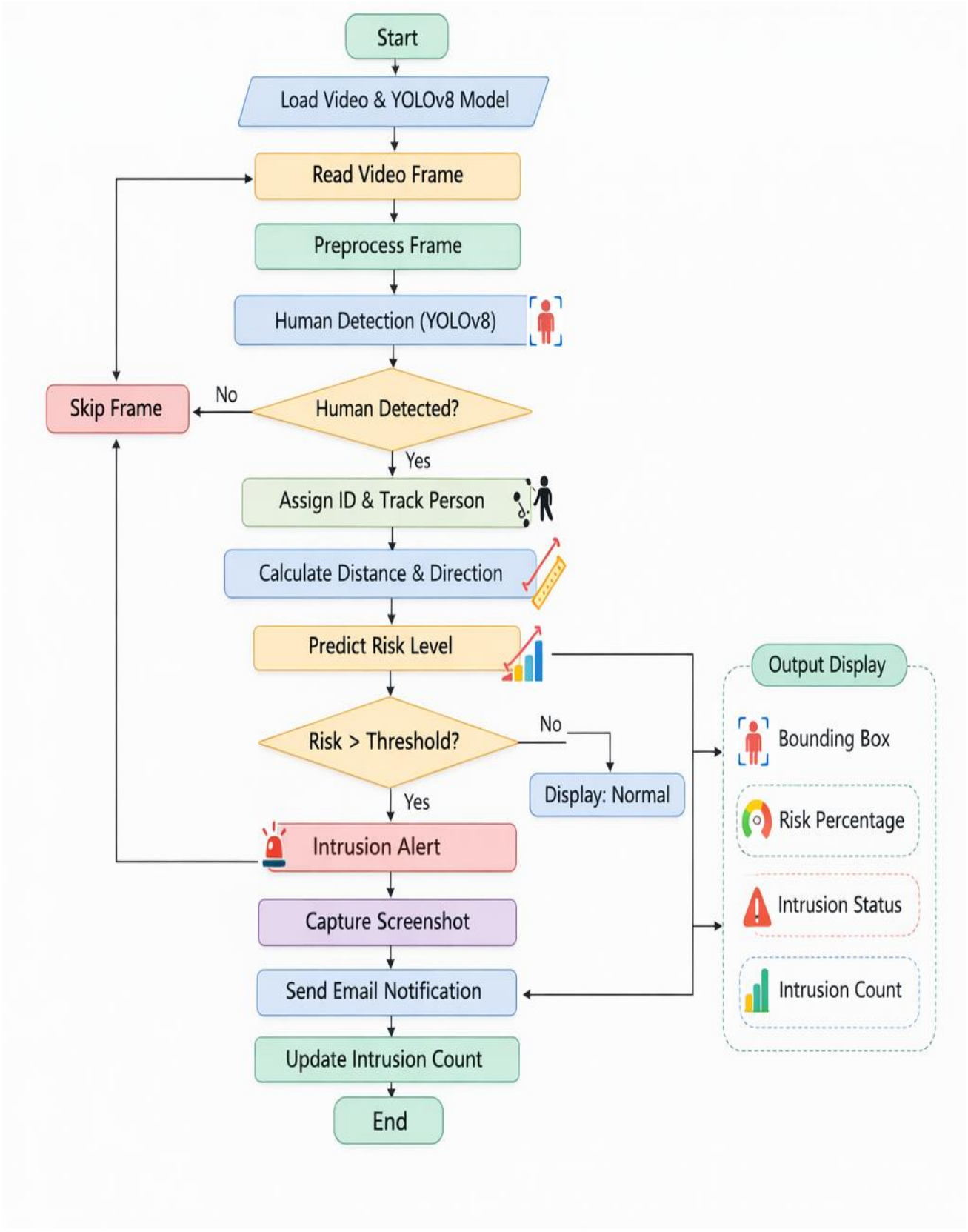


Fig 1: *System Flow Diagram*

4.4 DATA FLOW DIAGRAM

The Data Flow Diagram (DFD) represents how data moves between different components of the system. It illustrates the flow of information from the input stage to the output stage through various processing modules.

The first stage of the system involves capturing video frames from the input source. These frames are sent to the preprocessing module where they are resized and prepared for analysis.

The processed frames are then passed to the human detection module. This module uses the YOLOv8 model to detect individuals present in the frame and extract their positional information.

The detected object data is forwarded to the movement analysis module. This module evaluates the position of each detected individual relative to the restricted boundary and calculates movement direction and distance.

The intrusion prediction module uses this information to estimate the probability of intrusion. If the system determines that an intrusion has occurred, the alert generation module is activated.

The alert module captures an image of the intrusion event and sends an email notification to the authorized user. The system also updates the intrusion counter and displays the results on the monitoring screen.

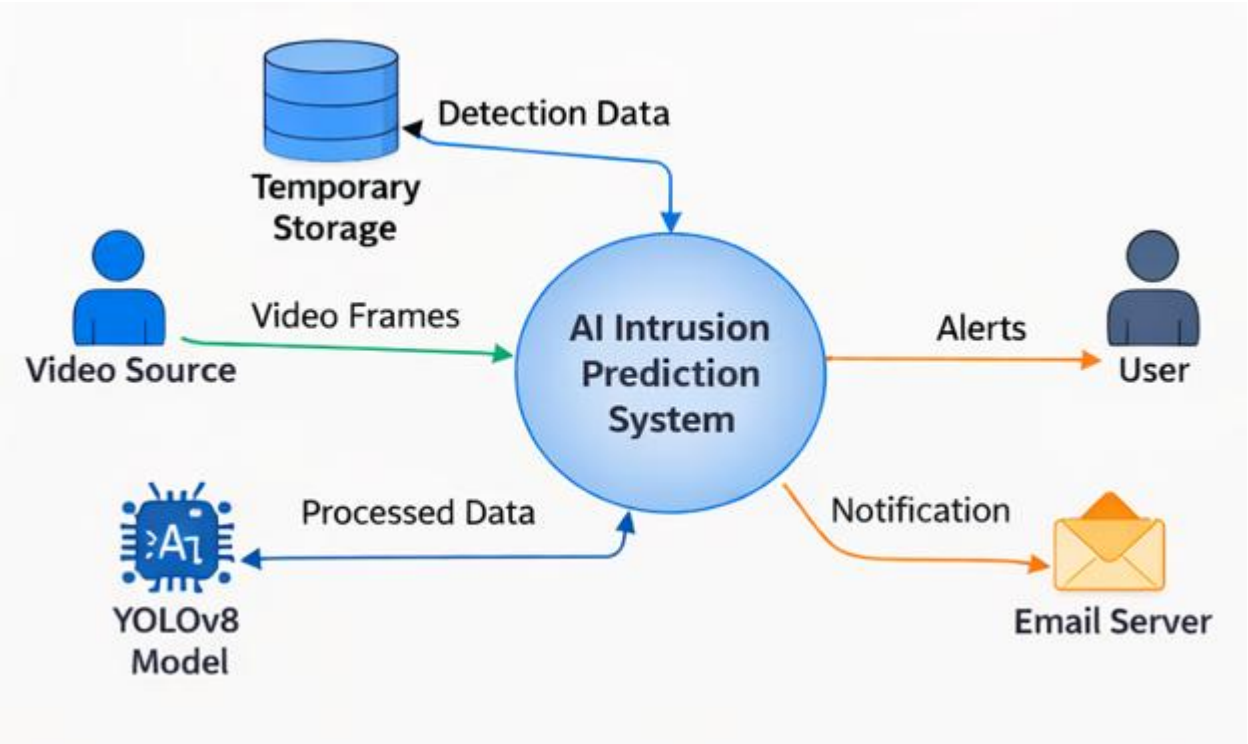


Fig 2: *Data Flow Diagram (Level 0)*

4.5 ENTITY RELATIONSHIP DIAGRAM

The Entity Relationship (ER) model represents how data is structured and related in the **AI-Based Intrusion Prediction System**. It illustrates the relationship between detected individuals, movement data, intrusion events, and alert notifications generated by the system. The ER model helps organize the data so that detection information, movement behavior, and intrusion alerts can be stored and managed efficiently. Through this model, the system can monitor detected persons, analyze their movement patterns, predict intrusion risks, and generate alerts when unauthorized entry occurs.

1. Person Entity

The **Person** entity represents individuals detected in the surveillance video. Each detected person is assigned a unique identifier by the system to track their movement across video frames. The system continuously records the position of each detected person and analyzes their behavior relative to the restricted boundary.

Attributes

Person_ID, X_Position, Y_Position, Frame_Number.

2. Movement Data Entity

The **Movement Data** entity stores information related to the movement behavior of detected individuals. It records the position changes of each person over time and calculates parameters such as distance from the restricted boundary and direction of movement.

Attributes

Person_ID, Distance_From_Boundary, Movement_Direction, Timestamp.

3. Intrusion Event Entity

The **Intrusion Event** entity records confirmed intrusion incidents detected by the system. When a person enters the restricted area, the system logs the event along with relevant details such as the detected person ID, calculated risk level, and captured screenshot.

Attributes

Event_ID, Person_ID, Risk_Level, Intrusion_Status, Screenshot_Path.

4. Alert Notification Entity

The **Alert Notification** entity stores information related to alerts generated by the system. When an intrusion occurs, the system sends an email notification to the authorized user. This entity records whether the alert was successfully sent and the timestamp of the notification.

Attributes

Alert_ID, Event_ID, Email_Status, Timestamp.

RELATIONSHIPS

A Person can generate multiple Movement Data records as their movement is tracked across multiple frames.

A Person may also trigger one or more Intrusion Events if they enter the restricted area.

Each Intrusion Event generates an Alert Notification which sends an email alert to the authorized user.

Therefore, the relationships between entities are structured as follows:

- **One Person → Many Movement Data**
- **One Person → Many Intrusion Events**
- **One Intrusion Event → One Alert Notification**

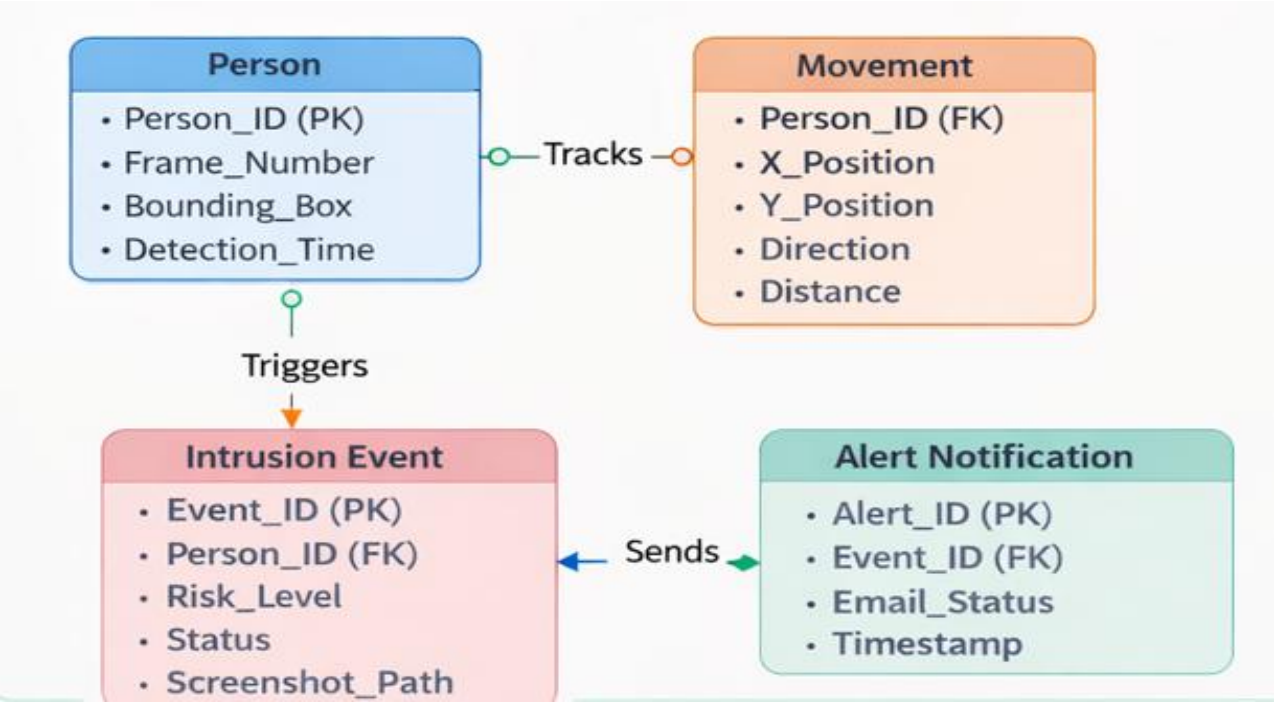


Fig 3 : *Diagram Representing Entity Relationship*

4.6 OUTPUT DESIGN

Output design describes how the results of the AI-Based Intrusion Prediction System are presented to the user. The system displays processed video frames along with detection results and prediction information. Visual indicators such as bounding boxes, risk percentage, and intrusion alerts help the user easily monitor the restricted area and quickly identify potential intrusion events.

1. Human Detection Display

The system detects people present in the video frame using the YOLOv8 object detection model. Detected individuals are highlighted using bounding boxes, making them easily visible to the user. Each detected person is also assigned a unique ID for tracking.

2. Risk Prediction Level

The system calculates a risk percentage based on the distance and movement of the detected person relative to the restricted boundary. This risk value indicates the likelihood of intrusion and helps provide an early warning before the boundary is crossed.

3. Intrusion Status Notification

When a person enters the restricted zone, the system immediately displays an intrusion alert message on the screen. This notification helps the user quickly recognize unauthorized access in the monitored area.

4. Intrusion Counter

The system maintains a counter that records the total number of intrusion events detected during monitoring. Each confirmed intrusion increases the counter, allowing users to track how many intrusion attempts have occurred.

5. Screenshot Capture

When an intrusion is detected, the system automatically captures a screenshot of the event. This image is stored for record purposes and can be used as visual evidence of the intrusion.

6. Email Alert Notification

The system sends an automated email alert to the authorized user when an intrusion occurs. The email contains a captured screenshot of the intrusion event, allowing the user to receive immediate notification even when not monitoring the screen.

5 SYSTEM TESTING AND IMPLEMENTATION

System testing and implementation are important stages in the development of any software system. Testing ensures that the developed system functions correctly, meets the intended objectives, and operates without errors under different conditions. Implementation involves deploying the system and verifying that it performs as expected in the operational environment.

For the AI-Based Intrusion Prediction System, testing was conducted to evaluate the performance of different modules including video input processing, human detection, object tracking, risk prediction, and alert generation. Each module was tested individually and then integrated to verify the complete system functionality.

The system was tested using multiple video samples containing different scenarios such as individuals approaching the restricted area, individuals crossing the boundary, and individuals moving away from the restricted zone. These tests helped evaluate the accuracy and reliability of the system in predicting and detecting intrusion events.

Testing also ensured that the alert generation mechanism worked correctly by capturing screenshots and sending email notifications to the authorized user when an intrusion occurred.

5.1 SYSTEM TESTING

System testing is the process of checking whether the complete system works correctly according to the specified requirements. It ensures that all modules function properly together and helps identify any errors before deployment.

1. Functional Testing

Functional testing verifies whether each module of the system performs its intended function correctly. In this project, the following components were tested:

- Video frame capture and processing
- Human detection using YOLOv8
- Object tracking across frames
- Risk prediction based on movement behavior
- Intrusion detection when boundary is crossed
- Email alert generation

Each module was tested independently to ensure correct functionality before integrating them into the complete system.

2. **Intrusion Detection Testing**

Intrusion detection testing was conducted to verify whether the system correctly identifies unauthorized entry into the restricted area.

Different video samples were used where individuals entered the restricted boundary. The system successfully detected the intrusion events and displayed appropriate alerts. The intrusion counter was updated correctly for each detected event.

3. **Intrusion Prediction Testing**

Prediction testing focused on evaluating the system's ability to predict potential intrusion before the restricted boundary was crossed.

During testing, individuals approaching the restricted area caused the system to gradually increase the risk level based on movement direction and distance from the boundary. This confirmed that the system could identify suspicious approach behavior and generate early warnings.

4. **Alert Notification Testing**

The alert generation mechanism was tested to verify that the system sends email notifications when intrusion occurs.

When a person entered the restricted area, the system captured a screenshot of the frame and successfully sent an email alert to the authorized user. The email contained the captured image as evidence of the intrusion event.

5. **Performance Testing**

Performance testing was conducted to evaluate the system's ability to process video frames efficiently. The system was tested with continuous video input, and it successfully maintained real-time processing without significant delays. The use of the YOLOv8 model enabled efficient detection and tracking of individuals in video frames.

5.2 SYSTEM IMPLEMENTATION

1. System Initialization

The system begins by loading the YOLOv8 deep learning model used for human detection. Required libraries such as OpenCV, NumPy, and other modules are initialized. The video input source is then loaded, and the application window for displaying system output is created. This step prepares the system environment for real-time intrusion monitoring.

2. Video Frame Capture

The system continuously reads frames from the input video source or surveillance camera. Each frame is captured and passed to the processing stage for analysis. These frames act as the main input data used by the system to detect and track individuals in the monitored area.

3. Human Detection and Tracking

The captured frames are processed using the YOLOv8 object detection model to identify human presence. Once a person is detected, the system generates bounding boxes around the detected individual and assigns a unique ID for tracking. The tracking mechanism allows the system to monitor the movement of individuals across multiple frames.

4. Restricted Boundary Definition

The system allows the user to define a restricted zone using a polygon boundary on the video frame. This boundary represents the protected area that should not be entered by unauthorized individuals. The system continuously checks whether detected persons are approaching or entering this defined region.

5. Intrusion Risk Prediction

The system analyzes the movement of detected individuals relative to the restricted boundary. It calculates parameters such as distance from the boundary, movement direction, and approach consistency. Based on these factors, the system calculates a risk level that predicts the likelihood of intrusion.

6. Intrusion Detection and Alert Generation

When a detected individual enters the restricted zone, the system confirms an intrusion event. At this stage, the system captures a screenshot of the frame and sends an automated email alert to the authorized user. The intrusion count is updated, and the system continues monitoring for further events.

7. Real-Time Output Display

The system displays real-time monitoring results on the screen. Bounding boxes are shown around detected persons, along with the predicted risk percentage and intrusion status. The system also displays the total number of intrusion events detected during operation.

6. CONCLUSION

The AI-Based Intrusion Prediction System developed in this project demonstrates how artificial intelligence and computer vision technologies can significantly enhance modern surveillance systems. Traditional surveillance systems mainly rely on manual monitoring of CCTV cameras, which can lead to delayed responses and human errors due to fatigue or distraction. The proposed system addresses these limitations by introducing automated detection and prediction capabilities.

The system successfully integrates deep learning techniques, specifically the YOLOv8 object detection model, to detect human presence within surveillance video frames. By combining object detection with movement tracking and behavioral analysis, the system is able to monitor individuals approaching a restricted area and evaluate their potential intention to enter that area.

One of the key contributions of this system is its ability to predict intrusion before the restricted boundary is crossed. The system analyzes factors such as distance from the boundary, direction of movement, and consistency of approach across multiple frames. Based on this analysis, it calculates a risk level that represents the probability of intrusion.

When an intrusion event occurs, the system automatically captures a screenshot of the frame and sends an email notification to the authorized user. This alert mechanism ensures that security personnel are informed immediately and can respond quickly to potential threats.

The system also provides real-time visual output including bounding boxes around detected individuals, predicted risk percentage, intrusion status, and total intrusion count. This real-time feedback helps improve situational awareness and allows users to monitor surveillance activity more effectively.

Overall, the proposed system demonstrates that AI-powered surveillance can improve security monitoring by reducing reliance on manual observation while increasing detection accuracy and response speed. The system can be applied in various environments such as industrial facilities, warehouses, restricted campuses, and other security-sensitive locations.

7. FUTURE ENHANCEMENT

The AI-Based Intrusion Prediction System can be further improved in several ways to enhance its functionality, scalability, and real-world usability. Some possible future enhancements are listed below.

1. Live Camera Integration

The current system processes offline video input for testing purposes. In the future, the system can be integrated with live CCTV or IP camera feeds to enable real-time monitoring of restricted areas.

2. Multi-Camera Support

The system can be extended to support multiple surveillance cameras. This would allow large areas to be monitored simultaneously and improve overall security coverage.

3. Mobile Notification System

In addition to email alerts, the system can be enhanced to send notifications through mobile applications or SMS alerts, ensuring faster response to intrusion events.

4. Face Recognition Integration

Face recognition technology can be incorporated to identify authorized personnel and distinguish them from unauthorized individuals. This feature would reduce false alarms and improve system accuracy.

5. Cloud-Based Monitoring

A cloud-based dashboard can be developed to store surveillance data and allow remote monitoring from different locations. This would improve accessibility and system management.

6. IoT Security Integration

The system can be connected with IoT devices such as alarms, automated lights, or access control systems to trigger immediate responses during intrusion events.

7. Advanced Behavioral Analysis

Future versions of the system could include more advanced machine learning models capable of analyzing suspicious behavior patterns and predicting intrusion attempts with greater accuracy.

1. BIBLIOGRAPHY

1. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016).
   You Only Look Once: Unified, Real-Time Object Detection.
   Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
2. Bochkovskiy, A., Wang, C. Y., & Liao, H. Y. M. (2020).
   YOLOv4: Optimal Speed and Accuracy of Object Detection.
   arXiv preprint arXiv:2004.10934.
3. Ultralytics. (2024).
   Ultralytics YOLOv8 Documentation.
   Available: https://docs.ultralytics.com
4. Bradski, G., & Kaehler, A. (2008).
   Learning OpenCV: Computer Vision with the OpenCV Library.
   O'Reilly Media.
5. Szeliski, R. (2022).
   Computer Vision: Algorithms and Applications.
   Springer.
6. Russell, S., & Norvig, P. (2021).
   Artificial Intelligence: A Modern Approach (4th Edition).
   Pearson Education.
7. OpenCV Development Team. (2024).
   OpenCV Documentation – Open Source Computer Vision Library.
   Available: https://opencv.org
8. Python Software Foundation. (2024).
   Python Programming Language Documentation.
   Available: https://www.python.org

SOFTWARE TOOLS AND DEVELOPMENT PLATFORMS

1. Microsoft Corporation. (2024).
   Visual Studio Code Documentation – Development Environment for Software Applications.
   Available: https://code.visualstudio.com

2. Ultralytics. (2024).
   YOLOv8 Object Detection Framework Documentation.
   Available: https://docs.ultralytics.com
3. OpenCV Organization. (2024).
   OpenCV Library Documentation.
   Available: https://opencv.org
4. Python Software Foundation. (2024).
   Python Programming Language Documentation.
   Available: https://www.python.org

9. APPENDIX

9.1 SOURCE CODE

The AI-Based Intrusion Prediction and Alert System is implemented using Python and integrates computer vision and deep learning techniques. The system performs human detection and tracking, intrusion risk analysis, and automated alert generation. The complete source code is included in this appendix for reference and future enhancement.

```
import cv2
import numpy as np
import time
import threading
import smtplib
import os
from email.message import EmailMessage
from ultralytics import YOLO
from collections import defaultdict, deque

# ---------------- EMAIL CONFIG ----------------
SENDER_EMAIL = "<configured_sender_email>"
RECEIVER_EMAIL = "<configured_receiver_email>"
APP_PASSWORD = "<hidden_for_security>"

# ---------------- MODEL ----------------
model = YOLO("yolov8n.pt")

# ---------------- VIDEO CONFIG ----------------
cap = cv2.VideoCapture("input_video.mp4")
fps = cap.get(cv2.CAP_PROP_FPS)
fps = fps if fps > 0 else 30
dt = 1 / fps
delay = int(1000 / fps)

WINDOW = "AI Intrusion Prediction System"
cv2.namedWindow(WINDOW, cv2.WINDOW_NORMAL)
```

```
# ---------------- BOUNDARY ----------------
boundary_points = []

def draw_boundary(event, x, y, flags, param):
    if event == cv2.EVENT_LBUTTONDOWN:
        boundary_points.append((x, y))

cv2.setMouseCallback(WINDOW, draw_boundary)

# ---------------- TEMP STATE ----------------
track_history = defaultdict(lambda: deque(maxlen=5))
risk_memory = defaultdict(float)
approach_consistency = defaultdict(float)

in_session = defaultdict(bool)
alert_sent = defaultdict(bool)
exit_timer = defaultdict(float)

intrusion_count = 0
os.makedirs("screenshots", exist_ok=True)

# ---------------- EMAIL THREAD ----------------
def send_alert(frame_copy, pid):

    def worker():
        try:
            ts = time.strftime("%Y%m%d_%H%M%S")
            img_path = f"screenshots/intrusion_{pid}_{ts}.jpg"
            cv2.imwrite(img_path, frame_copy)

            msg = EmailMessage()
            msg["Subject"] = "Intrusion Detected"
            msg["From"] = SENDER_EMAIL
            msg["To"] = RECEIVER_EMAIL
            msg.set_content("A person has entered the restricted area.")
```

```
            with open(img_path, "rb") as f:
                msg.add_attachment(
                    f.read(),
                    maintype="image",
                    subtype="jpg",
                    filename=os.path.basename(img_path)
                )

            with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
                server.login(SENDER_EMAIL, APP_PASSWORD)
                server.send_message(msg)

        except Exception as e:
            print("Email error:", e)

    threading.Thread(target=worker, daemon=True).start()

# ---------------- GEOMETRY ----------------
def point_inside_polygon(point, polygon):
    if len(polygon) < 3:
        return False
    return cv2.pointPolygonTest(np.array(polygon, np.int32), point, False) >= 0

def distance_to_polygon(point, polygon):

    def seg_dist(p, a, b):
        p, a, b = map(np.array, (p, a, b))
        ab = b - a
        t = np.clip(np.dot(p - a, ab) / (np.dot(ab, ab) + 1e-6), 0, 1)
        return np.linalg.norm(p - (a + t * ab))

    return min(
        seg_dist(point, polygon[i], polygon[(i + 1) % len(polygon)])
        for i in range(len(polygon))
    )
```

```
# ---------------- UI HELPERS ----------------
def draw_corner_box(img, x1, y1, x2, y2, color, thickness=2, corner=18):

    cv2.line(img, (x1, y1), (x1 + corner, y1), color, thickness)
    cv2.line(img, (x1, y1), (x1, y1 + corner), color, thickness)

    cv2.line(img, (x2, y1), (x2 - corner, y1), color, thickness)
    cv2.line(img, (x2, y1), (x2, y1 + corner), color, thickness)

    cv2.line(img, (x1, y2), (x1 + corner, y2), color, thickness)
    cv2.line(img, (x1, y2), (x1, y2 - corner), color, thickness)

    cv2.line(img, (x2, y2), (x2 - corner, y2), color, thickness)
    cv2.line(img, (x2, y2), (x2, y2 - corner), color, thickness)

def draw_label_panel(img, x1, y1, lines, color):

    panel_h = 18 * len(lines)
    panel_w = 220

    overlay = img.copy()

    cv2.rectangle(
        overlay,
        (x1, y1 - panel_h - 6),
        (x1 + panel_w, y1),
        (20, 20, 20),
        -1
    )

    img[:] = cv2.addWeighted(overlay, 0.6, img, 0.4, 0)

    for i, text in enumerate(lines):
```

```
        cv2.putText(
            img,
            text,
            (x1 + 8, y1 - panel_h + 18 * i),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.45,
            color,
            1,
            cv2.LINE_AA
        )

# ---------------- MAIN LOOP ----------------
while True:

    ret, frame = cap.read()
    if not ret:
        break

    frame = cv2.resize(frame, (960, 540))
    overlay = frame.copy()

    results = model.track(
        frame,
        persist=True,
        conf=0.35,
        classes=[0],
        tracker="bytetrack.yaml",
        verbose=False
    )

    if len(boundary_points) >= 3:
        cv2.fillPoly(overlay, [np.array(boundary_points)], (0, 0, 255))
        frame = cv2.addWeighted(overlay, 0.25, frame, 0.75, 0)

    global_prediction = 0
```

```
for r in results:

    if r.boxes.id is None:
        continue

    for box, pid in zip(r.boxes.xyxy, r.boxes.id):

        pid = int(pid)
        x1, y1, x2, y2 = map(int, box)

        cx = (x1 + x2) // 2
        foot = (cx, y2)

        track_history[pid].append(foot)

        if len(boundary_points) < 3:
            continue

        d_now = distance_to_polygon(foot, boundary_points)
        inside = point_inside_polygon(foot, boundary_points)

        approach = 0

        if len(track_history[pid]) >= 2:
            prev = track_history[pid][-2]
            d_prev = distance_to_polygon(prev, boundary_points)
            approach = np.clip((d_prev - d_now) / max(d_prev, 1), -1, 1)

        if approach > 0.02:
            approach_consistency[pid] = min(1.0, approach_consistency[pid] + 0.05)
        else:
            approach_consistency[pid] = max(0.0, approach_consistency[pid] - 0.03)

        dist_score = np.clip(1 - d_now / 400, 0, 1)
        eta_score = np.clip(1 - d_now / 300, 0, 1)
```

```
raw_intent = (
    0.4 * dist_score +
    0.3 * eta_score +
    0.3 * approach_consistency[pid]
)

risk_memory[pid] = 0.25 * raw_intent + 0.75 * risk_memory[pid]
risk = int(np.clip(risk_memory[pid] * 99, 0, 99))

if inside:
    risk = 100

    if not in_session[pid]:
        intrusion_count += 1
        in_session[pid] = True

    if not alert_sent[pid]:
        send_alert(frame.copy(), pid)
        alert_sent[pid] = True
        exit_timer[pid] = 0

else:
    exit_timer[pid] += dt

    if exit_timer[pid] > 2:
        in_session[pid] = False
        alert_sent[pid] = False

global_prediction = max(global_prediction, risk)

if risk < 40:
    color, status = (80, 200, 120), "LOW RISK"
elif risk < 70:
    color, status = (0, 215, 255), "APPROACHING"
else:
```

```
        color, status = (0, 80, 255), "HIGH RISK"

        draw_corner_box(frame, x1, y1, x2, y2, color)

        draw_label_panel(
            frame,
            x1,
            y1,
            [

                f"PERSON ID : {pid}",
                f"STATUS : {status}",
                f"RISK : {risk}%"
            ],
            color
        )

        cv2.circle(frame, foot, 4, (255, 0, 0), -1)

cv2.putText(
    frame,
    f"PREDICTION LEVEL : {global_prediction}%",
    (20, 35),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.9,
    (255, 255, 255),
    2
)
```

```
    cv2.putText(
        frame,
        f"TOTAL INTRUSIONS : {intrusion_count}",
        (700, 35),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (255, 255, 255),
        2
    )

    for p in boundary_points:
        cv2.circle(frame, p, 5, (255, 255, 255), -1)

    if len(boundary_points) >= 2:
        cv2.polylines(frame, [np.array(boundary_points)], True, (255,255,255), 2)



    cv2.imshow(WINDOW, frame)

    if cv2.waitKey(delay) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

8.2. SYSTEM OUTPUT SCREENSHOTS

1. Person Detection Without Boundary

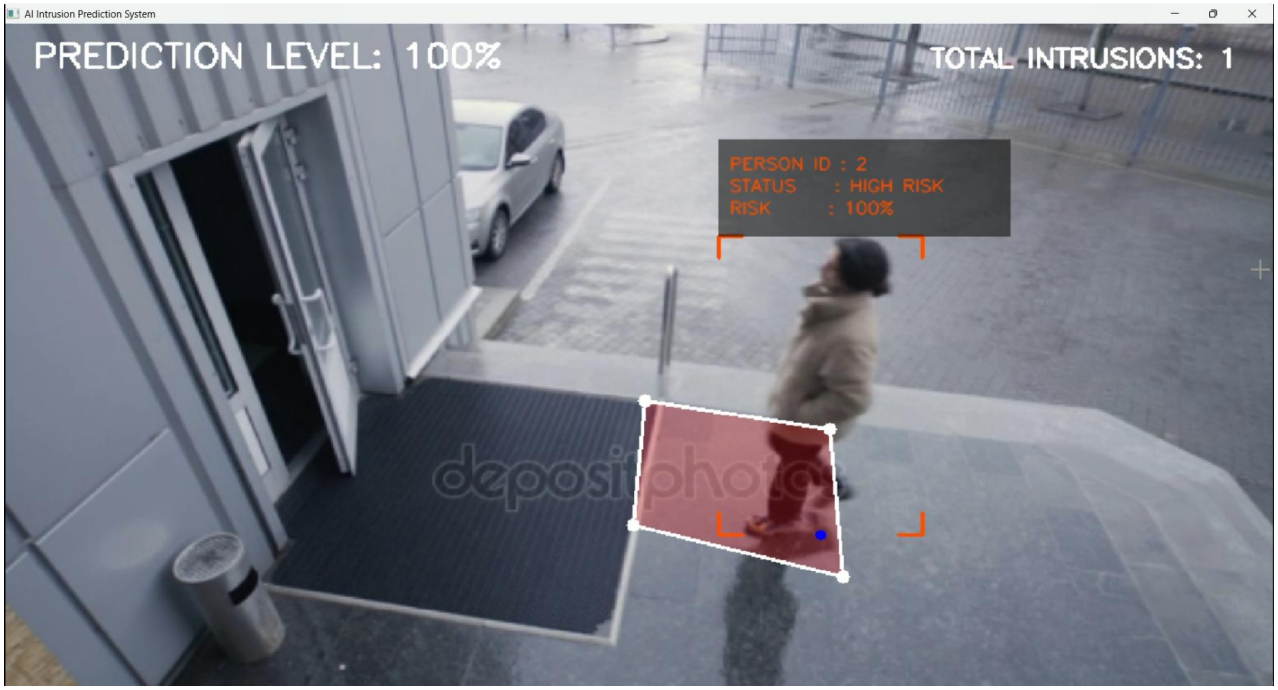
Shows human detection and ID assignment before defining the restricted boundary. The system displays *NO BOUNDARY* status with risk value set to 0%.

2. Intrusion Prediction During Approach

Illustrates prediction when a person approaches the restricted area. Risk percentage increases while the person moves closer to the boundary

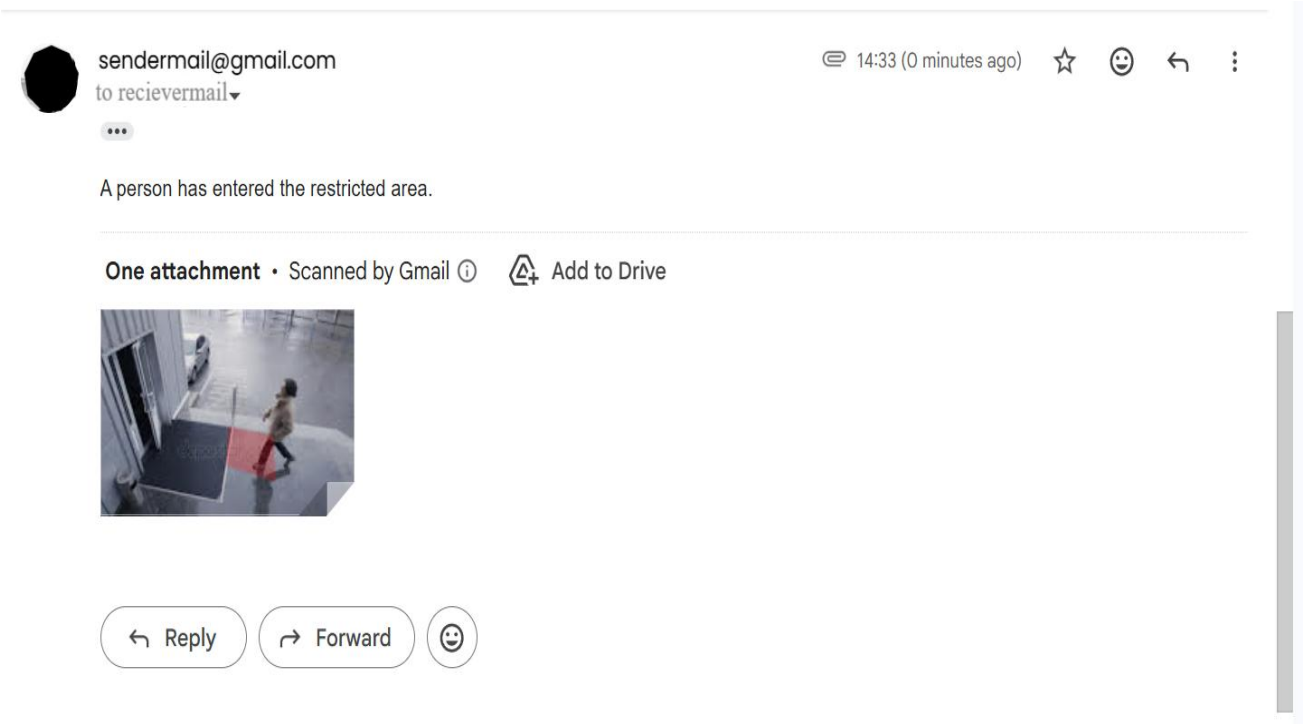
3. Confirmed Intrusion Detection

Shows a person entering the restricted zone. The risk level reaches 100%, and the intrusion count is incremented.



4. Risk Reset After Exit

Demonstrates reduction in risk after the person exits the restricted area, confirming proper temporal risk handling.

5. Email Alert Generated

Shows the automated email alert sent upon intrusion, with an attached snapshot captured at the time of boundary violation.